

# Μεταγλωττιστές 2024-25

Προσθήκη εντολών if και while στη γραμματική των αριθμητικών εκφράσεων

# Στόχος

- Η προσθήκη εντολών if και while στον συντακτικό αναλυτή και AST interpreter
  - Εκτός από τα ASSIGN και PRINT
- `if Lexpr Block_stmt`
- `while Lexpr Block_stmt`
  - block statement είναι
    - ή ένα μοναδικό `Stmt`
    - ή το σύνθετο `{ Stmt_list }`
  - Προς το παρόν θα παραλείψουμε τη δομή `else`

# Η νέα προτεινόμενη γραμματική

```
Stmt_list    → Stmt Stmt_list | ε
Stmt         → id = Lexpr | print Lexpr
              | if Lexpr Block_stmt | while Lexpr Block_stmt
Block_stmt   → Stmt | { Stmt_list }
Lexpr        → Lterm Lterm_tail
Lterm_tail   → or Lexpr | ε
Lterm        → Lfactor Lfactor_tail
Lfactor_tail → and Lterm | ε
Lfactor      → not Lfactor | Expr Rest
Rest         → Relop Expr | ε
Expr         → Term (Addop Term)*
Term_tail   → Addop Term Term_tail | ε
Term         → Factor (Multop Factor)*
Factor_tail → Multop Factor Factor_tail | ε
Factor       → ( Lexpr ) | id | number
Addop        → + | -
Multop       → * | /
Relop        → == | != | > | >= | < | <=
```

# FIRST & FOLLOW sets

| nonterminal       | first set                  | follow set                             |
|-------------------|----------------------------|----------------------------------------|
| Stmt_list         | id print <b>if while</b>   | } #                                    |
| Stmt              | id print <b>if while</b>   |                                        |
| <b>Block_stmt</b> | <b>{ id print if while</b> |                                        |
| Lexpr             | not ( id number            |                                        |
| Lterm_tail        | or                         | ) {} id print <b>if while</b> #        |
| Lterm             | not ( id number            |                                        |
| Lfactor_tail      | and                        | ) {} or id print <b>if while</b> #     |
| Lfactor           | not ( id number            |                                        |
| Rest              | == != > >= < <=            | ) {} and or id print <b>if while</b> # |
| Expr              | ( id number                |                                        |
| Term              | ( id number                |                                        |
| Factor            | ( id number                |                                        |
| Multop            | * /                        |                                        |
| Addop             | + -                        |                                        |
| Relop             | == != > >= < <=            |                                        |

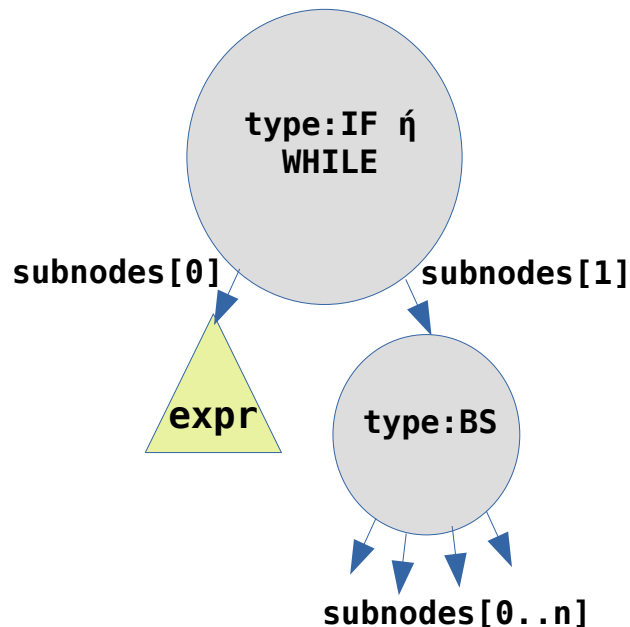
Νέο μη τερματικό σύμβολο: **Block\_stmt**

Νέα tokens: **if while { }**

# Δοκιμαστική είσοδος

```
a = 2 + 7.55*44
print a
if a-7!=3 or a<355 {
    b = 3*(a-99.01)
    it = 5
    while it>0 {
        print it+b*0.23
        it = it - 1
    }
}
```

# ASTnodes για νέες εντολές if και while



- **subnodes[0]** είναι το δένδρο που επιστρέφει η Lexpr() του if/while (AST λογικής/αριθμητικής έκφρασης)
- **subnodes[1]** είναι ο κόμβος που επιστρέφει η Block\_stmt() του if/while (με subnodes την ακολουθία των AST για τις εντολές του Block\_statement)

# Προσθήκες στον AST interpreter

- Στη μέθοδο `execute_statements()` προσθέστε τον χειρισμό των κόμβων τύπου IF και WHILE
  - Υπολογίστε την τιμή της έκφρασης E
    - Καλέστε την `evaluate_expression()` για το υποδένδρο `subnodes[0]`
    - Εάν/Όσο η επιστρεφόμενη τιμή είναι διάφορη του 0, τότε καλέστε την `execute_statements()` για τα `subnodes` του `subnodes[1]` (εκτέλεση εντολών που περιέχονται στο block statement BS)

# Προσθήκη του “else”

```
Stmt      → if Lexpr Block_stmt  
           | if Lexpr Block_Stmt else Block_Stmt  
           | ...
```

- Στόχος είναι να προστεθεί (προαιρετικά) το else στη δομή if
  - Μέχρι τώρα έχουμε υλοποιήσει μόνο τον κανόνα **if Expr Block\_stmt**
- Είναι η γραμματική LL(1);
  - Αρχικά θα πρέπει να φύγει ο «κοινός παράγοντας» (κοινό πρόθεμα **if Expr**) από τους δύο κανόνες if



# Προσθήκη του “else”

```
Stmt      → if Lexpr Block_stmt Rest_if  
          | ...  
Rest_if   → else Block_stmt | ε
```

- Μετά την απαλοιφή του κοινού προθέματος είναι η νέα γραμματική LL(1);
  - Ας βρούμε τα FIRST και FOLLOW sets για το **Rest\_if**

# FIRST/FOLLOW sets για το Rest\_if

| Σύνολο FOLLOW                                         | Σύνολο FIRST            | Κανόνες                                    |
|-------------------------------------------------------|-------------------------|--------------------------------------------|
| <code>#, else, },<br/>id, print,<br/>if, while</code> | <code>else<br/>ε</code> | <code>Rest_if → else Block_stmt   ε</code> |

- Υπάρχει σύγκρουση μεταξύ του FIRST και του FOLLOW set του μη τερματικού **Rest\_if** (**FIRST/FOLLOW conflict**)
  - Η γραμματική δεν είναι LL(1)
- Τι σημαίνει αυτό για την μέθοδο υλοποίησης που ακολουθούμε;

# Μια απόπειρα υλοποίησης...

## FIRST/FOLLOW conflict

Η γραμματική δεν είναι LL(1)!

```
def Rest_if(self):  
    if self.next_token == 'else':  
        # Rest_if → else Block_stmt  
        self.match('else')  
        self.Block_stmt()  
    elif self.next_token in ('else', None):  
        # Rest_if → ε  
        return  
    else:  
        raise ParseError
```

Πώς θα διαλέξουμε κανόνα όταν εμφανιστεί ένα else;

if expr if expr stmt1 else stmt2

Σε ποιο if ανήκει το else;

# Η ιδέα

```
Stmt -> if Lexpr Block_stmt (else Block_stmt)?  
      | ...
```

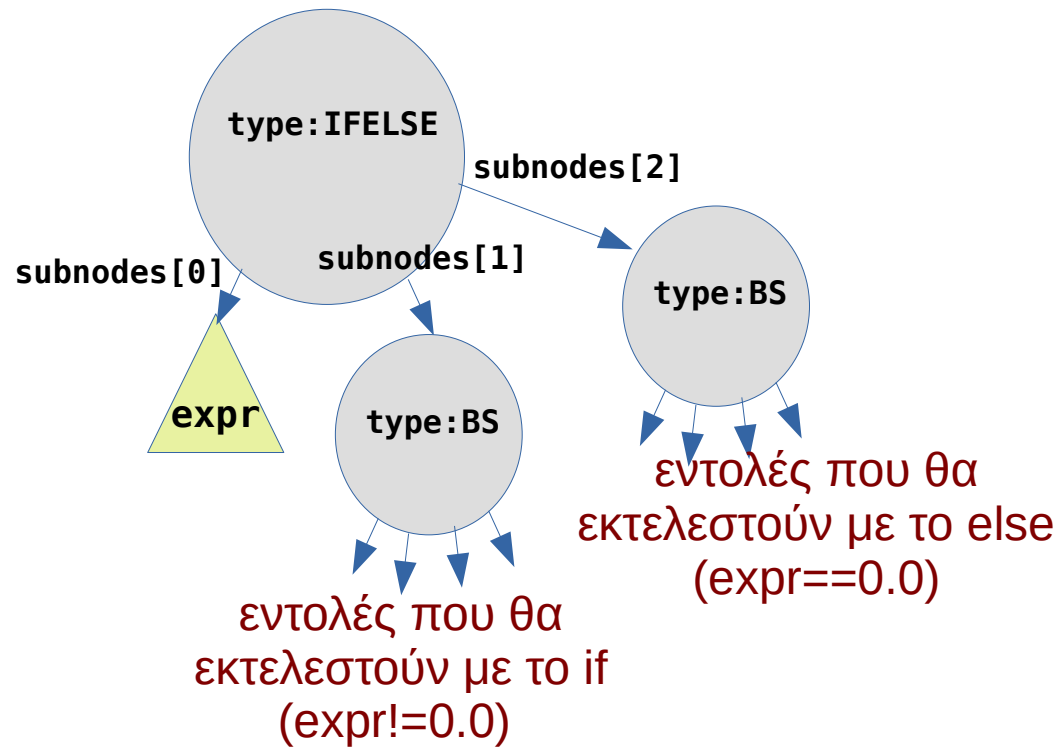
- **Εάν** μετά το if Lexpr Block\_stmt ακολουθεί else, τότε ταιριάζουμε κατευθείαν το else Block\_stmt
  - Συνεπώς το else θα συνδυαστεί με το τελευταίο if που έχουμε ήδη επεξεργαστεί, δηλ. το **κοντινότερο if**
- Επεκταμένη σύνταξη γραμματικής, **(...)?** = «προαιρετικά»

# Ο νέος κώδικας του if στο Stmt()

```
def Stmt(self):  
    ...  
    if self.next_symbol.token=='if':  
        # Stmt -> if Lexpr Block_stmt (else Block_stmt)?  
        self.match('if')  
        self.Lexpr()  
        self.Block_stmt()  
        # test if optional part (else Block_stmt) follows  
        if self.next_symbol.token=='else':  
            self.match('else')  
            self.Block_stmt()  
    ...
```

**Προσοχή:** με την αλλαγή αυτή το **else** προστίθεται στα follow sets των **Rest, Lfactor\_tail, Lterm\_tail**

# ASTNode για το if..else..



# Νέα δοκιμαστική είσοδος

```
a = 2 + 7.55*44
print a
if a-7!=3 or a<355 {
    b = 3*(a-99.01)
    it = 5
    while it>0 {
        print it+b*0.23
        it = it - 1
    }
}
else
    print a-3.14
```